

1. Build Your Own SSH

Alice and Bob work in different buildings on the same campus. They need to send files and run commands on each other's machines, but the only network between them is a shared office LAN that the IT team can monitor at any time, and they definitely do not want evil Dr. Ganguly to sniff their messages.

To solve this problem, they hire Lakshya and Sahil, the legendary Networking Engineers, to build a secure communication channel from scratch. During their glorious tenure as secretaries, they had already engineered such a beautiful and efficient proprietary TCP/IP stack that even C and Python allegedly modeled their standard socket libraries after it. Now, they need your help to take the next step: building their own Secure Shell system completely from scratch, without relying on any library that does the hard part.

Secure Shell (SSH) is the cryptographic network protocol that the entire tech industry relies on to securely log into remote machines, execute commands, and transfer files over unsecured networks. In this project, you are going to look under the hood and build the core mechanics of SSH completely from scratch using their custom TCP/IP stack.

IMPORTANT RULES:

- USE OF HIGH-LEVEL SECURE LIBRARIES (LIKE PARAMIKO OR ASYNCSH) IS NOT ALLOWED. ONLY USE BASIC SOCKETS AND CRYPTOGRAPHIC PRIMITIVES.
- **You must know how networking works.** A solid understanding of TCP/IP and how basic network sockets operate is strictly required to complete this project.
- **Version Control:** You must track your progress using GitHub. You are required to make a specific, clear commit after completing *each* milestone phase. You are highly encouraged to make as many other commits as you want in between these milestones.

System Requirements & Phases

Phase 1: Raw Sockets

- Write a server program and a client program that exchange messages over TCP.
- No encryption yet: just make the two machines talk.
- Run Wireshark during the exchange and capture a screenshot showing your plaintext message visible in the packet dump. This is the problem you are solving.

Phase 2: Symmetric Encryption

- Using AES-GCM, encrypt every message before it is sent and decrypt it after it is received.
- Both sides share a hardcoded key for this phase.
- Capture traffic again in Wireshark: the message should now be an unreadable AES-GCM payload.

Phase 3: Asymmetric Encryption and Key Exchange

- Generate an RSA-2048 keypair for the server.

- On connection, the server sends its public key to the client.
- The client generates a random 32-byte session key, encrypts it with the server's public key (RSA-OAEP), and sends it over. The server decrypts it with its private key.
- Replace the hardcoded key from Phase 2 with this newly shared session key.

Phase 4: Authentication

- The client generates its own RSA keypair. The client's public key is added to a file on the server called `authorized_keys.json`.
- After the encrypted channel is set up, the server sends the client a random 32-byte challenge.
- The client signs it with its private key (RSA-PSS). The server verifies the signature against the public key in `authorized_keys.json`.
- Match - access granted. No match - connection closed.

Phase 5: Remote Shell

- After authentication succeeds, run a loop: the client sends a command, the server executes it, and sends back the encrypted output.
- Basic system commands like `whoami`, `ls`, and `pwd` must work.

Inputs & Allowed Tools

- **Language:** Python or C.
- **Environment:** Two machines (or two terminal windows on the same machine).
- **Allowed Libraries:** Python standard library (socket, subprocess, os, json) and the cryptography pip package. If using C, you may use OpenSSL for the crypto primitives only.
- GPT is allowed to help you understand concepts. However, it is better to learn from other sources as it is more fun and you will know about more things. Additionally, if you cannot explain a line in your code during the demo, it wouldn't lead to a good impact on your performance during the interview.

Deliverables

- Working server program code.
- Working client program code.
- The `authorized_keys.json` file.
- Wireshark(or tcpdump) screenshots from Phase 1 (plaintext visible) and Phase 2 (encrypted, unreadable) side by side.

Submit a link to your GitHub repository showing your regular commits and your specific Milestone commits for each phase along with the images in a folder. A good file structure is appreciated.

Bonus Points: Awarded for utilizing tcpdump via the CLI instead of the Wireshark GUI, implementing perfect forward secrecy, or handling multiple concurrent clients.

POC: Sahil Agarwal, Lakshya Gupta